LABORATORY WORK #7

DB26. Pushkin Daniil

Task 1

```sql
-- Task 1: Create an index on the actual_departure column
CREATE INDEX idx_flights_actual_departure
ON flights (actual_departure_time);
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.03 rows=1 width=332) (actual time=0.014..0.015 rows=1.00 loops=1)
2	Filter: ((departing_airport_id = 1) AND (arriving_airport_id = 2))
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=30 read=2
7	Planning Time: 0.528 ms
8	Execution Time: 0.028 ms

Task 2

```sql
-- Task 2: Create a unique index for flight_no and scheduled_departure
CREATE UNIQUE INDEX idx_flights_unique_flight_departure
ON flights (flight_id, sch_departure_time);
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.03 rows=1 width=332) (actual time=0.008..0.009 rows=1.00 loops=1)
2	Filter: ((departing_airport_id = 1) AND (arriving_airport_id = 2))
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=18 read=1
7	Planning Time: 0.245 ms
8	Execution Time: 0.020 ms

Task 3

```sql
-- Task 3: Create a composite index on departure and arrival airports
CREATE INDEX idx_flights_route
ON flights (departing_airport_id, arriving_airport_id);
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.03 rows=1 width=12) (actual time=0.007..0.008 rows=1.00 loops=1)
2	Filter: ((departing_airport_id = 5) AND (arriving_airport_id = 10))
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning Time: 0.047 ms
6	Execution Time: 0.017 ms

Task 4

```sql
-- Task 4: Evaluate query performance with and without indexes
DROP INDEX IF EXISTS idx_flights_route;
EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departing_airport_id = 1 AND arriving_airport_id = 2;

CREATE INDEX idx_flights_route
ON flights (departing_airport_id, arriving_airport_id);
EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departing_airport_id = 1 AND arriving_airport_id = 2;
```

OUTPUT:

#	QUERY PLAN
1	width=332) (actual time=0.013..0.014 rows=1.00 loops=1)
2	Filter: ((departing_airport_id = 1) AND (arriving_airport_id = 2))
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=30 read=2
7	Planning Time: 0.551 ms
8	Execution Time: 0.027 ms

Task 5

```sql
-- Task 5: Check index usage in a query filtering by departure and arrival
EXPLAIN ANALYZE
SELECT flight_id, sch_departure_time
FROM flights
WHERE departing_airport_id = 5 AND arriving_airport_id = 10;
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.03 rows=1 width=12) (actual time=0.007..0.008 rows=1.00 loops=1)
2	Filter: ((departing_airport_id = 5) AND (arriving_airport_id = 10))
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning Time: 0.047 ms
6	Execution Time: 0.017 ms

Task 6

```sql
-- Task 6: Unique index for passport_number in Passengers
CREATE UNIQUE INDEX idx_passengers_passport
ON Passengers (passport_number);

-- Verify by checking system catalogs
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers' AND indexname = 'idx_passengers_passport';

-- Insert two new passengers to test uniqueness
INSERT INTO Passengers (first_name, last_name, date_of_birth, gender, country_of_citizenship,
country_of_residence, passport_number, created_at, updated_at)
VALUES ('John', 'Doe', '1990-01-01', 'Male', 'USA', 'USA', 'A1234567', CURRENT_TIMESTAMP, CURRE
NT_TIMESTAMP);
```

OUTPUT:

#	indexname	indexdef
1	idx_passengers_passport	CREATE UNIQUE INDEX idx_passengers_passport ON public.passengers USING btree (passport_number)

Task 7

```sql
-- Task 7: Index on multiple passenger details and filter check
CREATE INDEX idx_passengers_details
ON Passengers (first_name, last_name, date_of_birth, country_of_citizenship);

EXPLAIN ANALYZE
SELECT * FROM Passengers
WHERE country_of_citizenship = 'Philippines'
  AND EXTRACT(YEAR FROM date_of_birth) = 1984;
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on passengers (cost=0.00..1.05 rows=1 width=732) (actual time=0.025..0.026 rows=1.00 loops=1)
2	Filter: (((country_of_citizenship)::text = 'Philippines'::text) AND (EXTRACT(year FROM date_of_birth) = '1984'::numeric))
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=38 read=2
7	Planning Time: 0.535 ms
8	Execution Time: 0.046 ms

Task 8

```sql
-- Task 8: List indexes for Passengers and delete them
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';

DROP INDEX idx_passengers_passport;
DROP INDEX idx_passengers_details;
```

OUTPUT:

#	indexname	indexdef
1	passengers_pkey	CREATE UNIQUE INDEX passengers_pkey ON public.passengers USING btree (passenger_id)
2	idx_passengers_passport	CREATE UNIQUE INDEX idx_passengers_passport ON public.passengers USING btree (passport_number)
3	idx_passengers_details	CREATE INDEX idx_passengers_details ON public.passengers USING btree (first_name, last_name, date_of_birth, country_of_citizenship)

Task 9

```sql
-- Task 9: Compare execution plan before and after index on passenger_id in booking
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;

CREATE INDEX idx_booking_passenger_id ON booking (passenger_id);

EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
```

OUTPUT (BEFORE INDEX):

#	QUERY PLAN
1	Seq Scan on booking (cost=0.00..13.25 rows=1 width=278) (actual time=0.142..0.143 rows=2.00 loops=1)
2	Filter: (passenger_id = 100)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning Time: 0.050 ms
6	Execution Time: 0.153 ms

OUTPUT (AFTER INDEX):

#	QUERY PLAN
1	Seq Scan on booking (cost=0.00..1.05 rows=1 width=278) (actual time=0.008..0.009 rows=2.00 loops=1)
2	Filter: (passenger_id = 100)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=15 read=1
7	Planning Time: 0.384 ms
8	Execution Time: 0.018 ms

Task 10

```sql
-- Task 10: Composite index on booking_flight and Join query
CREATE INDEX idx_booking_flight_composite
ON Booking_flight (booking_id, flight_id);

EXPLAIN ANALYZE
SELECT f.flight_id, b.ticket_price
FROM booking b
JOIN Booking_flight bf ON b.booking_id = bf.booking_id
JOIN flights f ON bf.flight_id = f.flight_id
WHERE b.ticket_price > 9000;
```

OUTPUT:

#	QUERY PLAN
1	Hash Join (cost=5.25..17.49 rows=362 width=18) (actual time=0.099..0.101 rows=0.00 loops=1)
2	Hash Cond: (bf.flight_id = f.flight_id)
3	Buffers: shared hit=6
4	-> Nested Loop (cost=4.21..15.47 rows=362 width=18) (actual time=0.024..0.024 rows=0.00 loops=1)
5	Buffers: shared hit=5
6	-> Seq Scan on booking b (cost=0.00..1.05 rows=1 width=18) (actual time=0.012..0.013 rows=2.00 loops=1)
7	Filter: (ticket_price > '9000'::numeric)
8	Rows Removed by Filter: 2
9	Buffers: shared hit=1
10	-> Bitmap Heap Scan on booking_flight bf (cost=4.21..14.35 rows=7 width=8) (actual time=0.004..0.004 rows=0.00 loops=2)
11	Recheck Cond: (b.booking_id = booking_id)
12	Buffers: shared hit=4
13	-> Bitmap Index Scan on idx_booking_flight_composite (cost=0.00..4.21 rows=7 width=0) (actual time=0.002..0.002 rows=0.00 loops=2)
14	Index Cond: (booking_id = b.booking_id)
15	Index Searches: 2
16	Buffers: shared hit=4
17	-> Hash (cost=1.02..1.02 rows=2 width=4) (actual time=0.017..0.018 rows=2.00 loops=1)
18	Buckets: 1024 Batches: 1 Memory Usage: 9kB
19	Buffers: shared hit=1
20	-> Seq Scan on flights f (cost=0.00..1.02 rows=2 width=4) (actual time=0.008..0.008 rows=2.00 loops=1)

Task 11

```sql
-- Task 11: Index on status in flights (Selectivity test)
CREATE INDEX idx_flights_status ON flights (status);

EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Departed';
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Boarding';
```

OUTPUT (DEPARTED):

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.02 rows=1 width=332) (actual time=0.011..0.012 rows=1.00 loops=1)
2	Filter: ((status)::text = 'Departed'::text)
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=15 read=1
7	Planning Time: 0.310 ms
8	Execution Time: 0.024 ms

OUTPUT (BOARDING):

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.02 rows=1 width=332) (actual time=0.007..0.008 rows=1.00 loops=1)
2	Filter: ((status)::text = 'Boarding'::text)
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning Time: 0.064 ms
6	Execution Time: 0.017 ms

Task 12

```sql
-- Task 12: Selectivity on weight_in_kg
CREATE INDEX idx_baggage_weight ON Baggage (weight_in_kg);

EXPLAIN ANALYZE SELECT * FROM Baggage WHERE weight_in_kg > 49;
EXPLAIN ANALYZE SELECT * FROM Baggage WHERE weight_in_kg > 5;
```

OUTPUT (WEIGHT > 49):

#	QUERY PLAN
1	Seq Scan on baggage (cost=0.00..1.05 rows=1 width=36) (actual time=0.009..0.009 rows=0.00 loops=1)
2	Filter: (weight_in_kg > '49'::numeric)
3	Rows Removed by Filter: 4
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=19 read=1
7	Planning Time: 0.318 ms
8	Execution Time: 0.017 ms

OUTPUT (WEIGHT > 5):

#	QUERY PLAN
1	Seq Scan on baggage (cost=0.00..1.05 rows=1 width=36) (actual time=0.008..0.009 rows=2.00 loops=1)
2	Filter: (weight_in_kg > '5'::numeric)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning Time: 0.060 ms
6	Execution Time: 0.017 ms

Task 13

```sql
-- Task 13: LIKE queries and indexes on airport_name
CREATE INDEX idx_airport_name ON Airport (airport_name);

EXPLAIN ANALYZE SELECT * FROM Airport WHERE airport_name LIKE 'A%';
EXPLAIN ANALYZE SELECT * FROM Airport WHERE airport_name LIKE '%Airport';
```

OUTPUT ('A%'):

#	QUERY PLAN
1	Seq Scan on airport (cost=0.00..1.25 rows=1 width=492) (actual time=0.041..0.044 rows=3.00 loops=1)
2	Filter: ((airport_name)::text ~~ 'A%'::text)
3	Rows Removed by Filter: 17
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=30 read=5
7	Planning Time: 7.027 ms
8	Execution Time: 0.062 ms

OUTPUT ('%AIRPORT'):

#	QUERY PLAN
1	Seq Scan on airport (cost=0.00..1.25 rows=1 width=492) (actual time=0.009..0.014 rows=17.00 loops=1)
2	Filter: ((airport_name)::text ~~ '%Airport'::text)
3	Rows Removed by Filter: 3
4	Buffers: shared hit=1
5	Planning Time: 0.067 ms
6	Execution Time: 0.025 ms

Task 14

```sql
-- Task 14: Leftmost prefix rule on composite index (country, city)
CREATE INDEX idx_airport_location ON Airport (country, city);

EXPLAIN ANALYZE SELECT * FROM Airport WHERE country = 'Kazakhstan';
EXPLAIN ANALYZE SELECT * FROM Airport WHERE city = 'Almaty';
EXPLAIN ANALYZE SELECT * FROM Airport WHERE country = 'Kazakhstan' AND city = 'Almaty';
```

OUTPUT (COUNTRY = 'KAZAKHSTAN'):

#	QUERY PLAN
1	Seq Scan on airport (cost=0.00..1.25 rows=1 width=492) (actual time=0.011..0.011 rows=0.00 loops=1)
2	Filter: ((country)::text = 'Kazakhstan'::text)
3	Rows Removed by Filter: 20
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=19 read=1
7	Planning Time: 0.314 ms
8	Execution Time: 0.022 ms

OUTPUT (CITY = 'ALMATY'):

#	QUERY PLAN
1	Seq Scan on airport (cost=0.00..1.25 rows=1 width=492) (actual time=0.010..0.010 rows=0.00 loops=1)
2	Filter: ((city)::text = 'Almaty'::text)
3	Rows Removed by Filter: 20
4	Buffers: shared hit=1
5	Planning Time: 0.055 ms
6	Execution Time: 0.019 ms

OUTPUT (COUNTRY = 'KAZAKHSTAN' AND CITY = 'ALMATY'):

#	QUERY PLAN
1	Seq Scan on airport (cost=0.00..1.30 rows=1 width=492) (actual time=0.009..0.009 rows=0.00 loops=1)
2	Filter: (((country)::text = 'Kazakhstan'::text) AND ((city)::text = 'Almaty'::text))
3	Rows Removed by Filter: 20
4	Buffers: shared hit=1
5	Planning Time: 0.060 ms
6	Execution Time: 0.018 ms

Task 15

```sql
-- Task 15: Functional (Expression) Index on LOWER(last_name)
CREATE INDEX idx_passengers_lower_lastname ON Passengers (LOWER(last_name));

EXPLAIN ANALYZE SELECT * FROM Passengers WHERE LOWER(last_name) = 'irnis';
EXPLAIN ANALYZE SELECT * FROM Passengers WHERE last_name = 'Irnis';
```

OUTPUT (LOWER(LAST_NAME) = 'IRNIS'):

#	QUERY PLAN
1	Seq Scan on passengers (cost=0.00..1.04 rows=1 width=732) (actual time=0.013..0.013 rows=0.00 loops=1)
2	Filter: (lower((last_name)::text) = 'irnis'::text)
3	Rows Removed by Filter: 3
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=19 read=1
7	Planning Time: 0.332 ms
8	Execution Time: 0.025 ms

OUTPUT (LAST_NAME = 'IRNIS'):

#	QUERY PLAN
1	Seq Scan on passengers (cost=0.00..1.04 rows=1 width=732) (actual time=0.007..0.007 rows=0.00 loops=1)
2	Filter: ((last_name)::text = 'Irnis'::text)
3	Rows Removed by Filter: 3
4	Buffers: shared hit=1
5	Planning Time: 0.084 ms
6	Execution Time: 0.015 ms

Task 16

```sql
-- Task 16: Covering Index for Index-Only Scan
CREATE INDEX idx_booking_covering ON booking (passenger_id, ticket_price);

EXPLAIN ANALYZE SELECT ticket_price FROM booking WHERE passenger_id = 100;
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on booking (cost=0.00..1.05 rows=1 width=14) (actual time=0.009..0.010 rows=2.00 loops=1)
2	Filter: (passenger_id = 100)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=19 read=1
7	Planning Time: 0.278 ms
8	Execution Time: 0.018 ms

Task 17

```sql
-- Task 17: Unique index on airline_code and handling duplicates
SELECT airline_code, COUNT(*) FROM Airline GROUP BY airline_code HAVING COUNT(*) > 1;
CREATE UNIQUE INDEX idx_airline_code_unique ON Airline (airline_code);

EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departing_airport_id = 5 AND arriving_airport_id = 10;
```

OUTPUT:

#	QUERY PLAN
1	Seq Scan on flights (cost=0.00..1.03 rows=1 width=332) (actual time=0.007..0.008 rows=1.00 loops=1)
2	Filter: ((departing_airport_id = 5) AND (arriving_airport_id = 10))
3	Rows Removed by Filter: 1
4	Buffers: shared hit=1
5	Planning Time: 0.064 ms
6	Execution Time: 0.016 ms